

A seasoned Java programmer will expect a number of errors in the above code. But everything is fine with JavaScript. Figure 4 shows the output of the script *type.js*. Why does the variable *a* have the value 21 and variable *b* have the value 1? This is due to the implicit type conversion in JavaScript. The operator '-' performs just one function, a mathematical subtraction. When a string and a number are the operands of the '-' operator, JavaScript converts the string to a number. So in the line of code, *b* = '2' - 1, the string '2' is converted to the number 2, and the number 1 is subtracted from it to obtain 1. Notice that, here, the variable *b* contains a number, 1. Now let us look at what happens with the operator '+'. The operator '+' performs two functions, mathematical addition and string concatenation. When a string and a number are the operands of the '+' operator, instead of converting the string to a number, JavaScript converts the number to a string. So in the line of code *a* = '2' + 1, the number 1 is converted to the string '1', and the strings '2' and '1' are concatenated to give '21'. Notice that here, the variable 'a' contains a string, '21'. With that

```
21
1
123
33
```

Figure 4: Output of the JavaScript script *type.js*

knowledge, can you explain why the variable 'c' contains the string '123' and the variable 'd' contains the string '33'? I will give you a hint — the associativity of the operator '+' is what matters. The operator '+' has left-associativity. Hence in the line of code *c* = '1' + 2 + 3, the operation '1' + 2 is carried out first, resulting in the string '12' because '1' is a string. Then the expression becomes '12' + 3, resulting in the final string '123' stored in the variable *c*. But in the line of code *d* = 1 + 2 + '3', due to left-associativity of the operator '+', the first operation performed is 1+2, resulting in the number 3, because both 1 and 2 are numbers. Then the expression becomes 3 + '3', resulting in the final string '33' stored in the variable *d*. Imagine the horror of a Java programmer going through these results in his favourite programming language!

Lazy evaluation in Haskell

Haskell is a general-purpose purely functional programming language, which has a feature called lazy evaluation! Because of lazy evaluation, expressions are not evaluated immediately if they are just bound to variables. Instead, the evaluation of an expression is carried out only when its results are needed by other operations. For this reason, lazy evaluation is often called 'call-by-need'. Lazy evaluation enables infinite functions to be stored in Haskell lists. Figure 5 shows the processing of one such infinite list in the GHCi interactive environment. The list 'x' represents the infinite list of

natural numbers by the line of code *let x* = [1..]. Notice that 'x' is not evaluated at this point. The line of code *take 10 x* gives the list [1,2,3,4,5,6,7,8,9,10] as output (the first ten elements of the list *x*). But the third line of code *print x* will lead to the complete evaluation of the list 'x' resulting in printing the natural numbers (from which you have to forcefully exit). Notice that the lazy evaluation feature of Haskell is very powerful but at the same time has received some criticism from experts.

Multiple main methods in Java

When I first learned Java programming, I did a very poor job because I tried to learn some advanced features (Swing and AWT) without mastering the basics. Till recently, I was under the false impression that a Java program can contain only one main method. But then I got a rude shock — it is possible to have more than one main method in a Java program. For example, consider the Java program *AAA.java* shown below with two different classes *AAA* and *BBB*, each having one main method.

```
class AAA
{
    void disp()
    {
        System.out.println("Hello From
AAA...");
    }

    public static void main(String[] args)
```

```
Prelude> let x = [1..]
Prelude> take 10 x
[1,2,3,4,5,6,7,8,9,10]
Prelude> print x
[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28,29,30,31,32,33,34,35,36,37,38,39,40,41,42,43,44,45,46,47,48,49,50,51,52,53,54,55,56,57,58,59,60,61,62,63,64,65,66,67,68,69,70,71,72,73,74,75,76,77,78,79,80,81,82,83,84,85,86,87,88,89,90,91,92,93,94,95,96,97,98,99,100,101,102,103,104,105,106,107,108,109,110,111,112,113,114,115,116,117,118,119,120,121,122,123,124,125,126,127,128,129,130,131,132,133,134,135,136,137,138,139,140,141,142,143,144,145,146,147,148,149,150,151,152,153,154,155,156,157,158,159,160,161,162,163,164,165,166,167,168,169,170,171,172,173,174,175,176,177,178,179,180,181,182,183,184,185,186,187,188,189,190,191,192,193,194,195,196,197,198,199,200,201,202,203,204,205,206,207,208,209,210,211,212,213,214,215,216,217,218,219,220,221,222,223,224,225,226,227,228,229,230,231,232,233,234,235,236,237,238,239,240,241,242,243,244,245,246,247,248,249,250,251,252,253,254,255,256,257,258,259,260,261,262,263,264,265,266,267,268,269,270,271,272,273,274,275,276,277,278,279,280,281,282,283,284,285,286,287,288,289,290,291,292,293,294,295,296,297
```

Figure 5: Output of GHCi interactive environment